

MATEO HUANCHO MARIÑO

Backend Java Senior Developer | Full Stack Developer | Angular | Android | Microservices | IAM & BPMN

Lima, Peru · Open to remote (LATAM / EU / US time zones)

+51 918 076 179 · mhuanchom@gmail.com

linkedin.com/in/mhuancho08 · github.com/mhuancho · huancho.dev

PROFESSIONAL PROFILE

Senior Java backend developer with **8 years of experience** designing, building, modernising and stabilising enterprise software.

Specialised in Java, Spring Boot, Quarkus, Micronaut, Keycloak, microservices, contract-first APIs, hexagonal and clean architecture, IAM security, API integration, messaging and BPM automation with Camunda.

I currently work on **Java 21 with virtual threads, Spring Boot 4 and Quarkus 3.30**, on a multi-tenant electronic invoicing platform where I **created 12 of its 24 repositories from scratch**: the Keycloak identity layer, the shared contracts and resilience libraries, four API gateways and the asynchronous document pipeline.

My focus is backend and software architecture, with complementary experience in Angular and micro-frontends to work on decoupled, secure and maintainable platforms. I care about good practice, quality, traceability, explicit contracts, reusable components and the sustainable evolution of critical systems.

SPECIALISATION AND VALUE PROPOSITION

- **Backend architecture and domain design:** hexagonal architecture (ports & adapters), clean architecture, separation of use cases, ports/adapters, contracts, persistence and infrastructure.
- **Enterprise, multi-tenant microservices:** decoupled, reusable and secure domain services with contract-first REST APIs (OpenAPI), GraphQL, resilience, maintainability and operational scalability.
- **Advanced security and IAM:** Keycloak, OAuth2, OIDC, JWT, MFA/TOTP/OTP, custom SPIs, RBAC, claims, roles, groups, auditing, cross-cutting authorisation and centralised identity management.
- **Experience layer (BFF) and API gateways:** one gateway per channel with configuration-driven routes, global filters for tracing and per-tenant routing, cached tenant resolution and a stateless edge.
- **Resilience as a reusable component:** circuit breaker, retry, bulkhead and time limiter policy packaged as a library, plus a dedicated gateway that isolates legacy systems.
- **Reactive programming and decoupled frontend integration:** Spring WebFlux, Project Reactor, R2DBC, RxJS, Angular, micro-frontends with Native Federation, guards, interceptors, refresh token and dynamic portal composition.
- **Enterprise platform modernisation:** scalable and secure backend design, standardisation of reusable technical components, less duplication and sustainable evolution of critical systems.
- **BPMN processes, workflows and automation:** Camunda 7/8, workers, message correlation, state traceability, SLAs, N8N automation and low-code/no-code integration.
- **Integration processes and events:** Outbox Pattern, Kafka, RabbitMQ, Redis, domain events, synchronisation across APIs, BPM and external services, with transactional consistency and traceability.
- **Asynchronous processing with guarantees:** persisted idempotency, dead-letter queues with republishing, scheduled technical queue and chunked processing for bulk workloads.
- **Technical quality and good practice:** JUnit 5, Mockito, JaCoCo, SonarQube, SOLID/DRY, global error handling, standard responses, technical documentation, contracts and reusable corporate libraries.

TECHNICAL SKILLS

Languages: Java 8/11/17/21 (virtual threads, records, pattern matching), TypeScript, JavaScript, SQL/PLSQL, .NET, Node.js

Backend: Spring Boot 3 and 4, Spring Cloud, Spring Cloud Gateway, Spring Cloud Stream, WebFlux, Quarkus 3, Micronaut, Java EE / Jakarta EE, JAX-RS, MicroProfile, JPA, Hibernate, MyBatis, Panache, Micronaut Data JDBC

Architecture: hexagonal architecture, clean architecture, DDD, microservices, layered design, MVC, event-driven, CQRS, BFF / experience layer, framework-agnostic shared libraries, SOLID/DRY

APIs / Integration: REST, OpenAPI/Swagger, contract-first, GraphQL, Feign, WebClient, external services, SOAP, BPMN, workflows

Security: Keycloak 26, SPIs (Authenticator, FormAction, EmailSender), Admin API, FreeMarker themes, MFA/TOTP, OAuth2, OIDC, JWT, JWKS, RBAC, claims, roles, groups, multi-tenancy, API keys, CORS, just-in-time provisioning

Resilience: Resilience4j (circuit breaker, retry, bulkhead, time limiter), fallbacks, backpressure, timeouts, graceful shutdown, resilience testing

BPM / Automation: Camunda 7, Camunda 8, BPMN, DMN, workers, messages, timers, events, N8N

Data / Messaging: PostgreSQL, Oracle, SQL Server, MySQL, MongoDB, Redis, SingleStore, Kafka, RabbitMQ, Spring Cloud Stream, R2DBC, Flyway, Outbox Pattern, dead-letter queues, idempotency

Cloud / DevOps: Azure, AWS (S3, SES, CodeCommit), AKS, Docker, Kubernetes, Jenkins, GitLab CI/CD, Atlassian suite

Observability: Spring Boot Actuator, Micrometer, Prometheus, liveness/readiness probes, request correlation, structured logging

Frontend: Angular 11-21, RxJS, Signals, standalone components, Nx monorepo, Native Federation, TailwindCSS, React, keycloak-js

Mobile: Android Studio, Android SDK, REST API consumption, geolocation, maintenance and evolution of Android apps

Testing / Quality: JUnit 5, Mockito, RestAssured, JaCoCo, SonarQube, Karma, Jasmine, Vitest, Playwright

Complexless

Senior Java Backend Developer | May 2024 - Present

Responsible for designing and building enterprise solutions with a focus on backend, security, integration, architecture, technical standardisation, microservices, multi-tenancy, IAM, BPMN and decoupled backend/frontend platforms. Multi-tenant electronic invoicing platform: ~23 backend services and 4 Angular 21 applications.

1. IAM, multi-factor authentication and centralised user management

Stack / practices: Java 21, Quarkus 3.30, Spring Boot 4, Keycloak 26, OAuth2/OIDC/JWT, PostgreSQL and SQL Server, Angular 21, hexagonal architecture, virtual threads

- Solution to strengthen authentication, authorisation, user administration and the sign-in experience on a multi-customer enterprise platform.
- Built decoupled backend services for user, profile, permission and authorisation management, separating domain, use cases, ports, adapters and contracts.
- Extended Keycloak with custom SPIs to add corporate validation to the login flow, including mandatory company tax-ID validation and per-customer access rules.
- Integrated legacy users with just-in-time provisioning over JDBC, supporting MD5/SHA-256 password verification with constant-time comparison to reduce timing-attack risk.
- Implemented TOTP/OTP MFA with QR enrolment, multi-device support, internationalisation and accessibility.
- Standardised JWT claims into authorities and roles consumable by every microservice, enabling consistent authorisation across applications.
- **Added a FormAction SPI for secondary-user pre-registration** with company validation, duplicate control and custom messages, plus an **EmailSenderProvider SPI** so all identity-provider email goes through the platform's mail service instead of a separate SMTP server.
- **Built two FreeMarker login themes** (Keycloak 26) with QR and manual TOTP enrolment, multi-device OTP selection, **9 locales** and **branding resolved per access domain**, so each company sees its own visual identity.
- **Moved full-company migration off the request thread onto virtual threads (Java 21)** and automated role and group assignment through the Admin API based on contracted products.

2. Modular multi-tenant enterprise ecosystem with Quarkus

Stack / practices: Java 21, Quarkus 3.30/3.31, PostgreSQL and SQL Server, Panache, Flyway, Keycloak, OpenAPI, corporate libraries

- Built an ecosystem of decoupled services for administration, onboarding, enterprise authorisation and multi-customer operations.
- Applied domain-driven and layered design to isolate business rules, persistence, security, REST contracts and technical adapters.
- Implemented reusable patterns for pagination, metadata, typed responses, validation, global error handling and cross-cutting security.
- Designed corporate libraries for `ApiResponse<T>`, pagination, exceptions, JWT validation, claim-to-authority mapping and workflow base components.
- Reduced duplication across services and raised technical consistency in authentication, authorisation, contracts, errors and response structure.
- **Made the corporate library deliberately framework-agnostic:** the multi-tenant context is built from JWT claims with no dependency on Quarkus or Spring, so services on both stacks share the same contract.
- **Packaged the security chain as opt-in auto-configuration behind a single annotation:** JWKS-based JWT validation, issuer verification, resource_access role mapping, configurable public paths and required role — removing duplicated security configuration from more than 10 services.
- **Led the evolution of the security API to a v2** in a new repository, replacing manual JDBC with Panache entities, versioning the schema with **Flyway**, separating the identity and mail clients and adding scheduled tasks — without interrupting v1 in production.

3. Security and ingestion layer for electronic documents

Stack / practices: Java 21, Quarkus 3.30, JAX-RS, virtual threads, API keys, contract-first OpenAPI, PostgreSQL and SQL Server

- Built the authenticated entry point that receives electronic documents and forwards them to the processing systems, with **per-company API keys** for external integrators and OIDC tokens for the portal.
- Implemented **streaming multipart upload** to a restricted-permission temporary file instead of buffering in memory, removing memory consumption proportional to package size.
- Ran the proxy resources on **virtual threads**, so I/O waits against remote systems no longer block the request pool.
- Unified error handling with exception mappers so every failure returns the same response contract, and versioned the **OpenAPI contract inside the repository** as the source of truth for integrators.

4. Experience layer (BFF) and multi-tenant API gateways

Stack / practices: Spring Boot 4, Spring Cloud Gateway (WebFlux and WebMVC), reactive WebClient, OAuth2 Resource Server, Micrometer/Prometheus

- Created **four gateways, one per audience** (internal portal, public channel, external integrators and resilience) instead of a single gateway: each channel has its own CORS, timeouts, exposed surface and authentication method.
- Declared routes through configuration rather than code, so adding a destination does not require a rebuild.
- Developed custom global filters for **tracing** (correlation header propagation), **per-company downstream routing** and context headers for domain services.

- Kept the **edge free of any database**: company resolution is an HTTP call to the configuration service, backed by a **bounded TTL cache with single-flight lookups**, so an entry expiring at peak time does not trigger a stampede of identical queries.
- Tuned connection pools, timeouts and graceful shutdown, and restricted Actuator to health, info and prometheus with liveness/readiness probes.

5. Cross-cutting resilience: circuit-breaker library and isolation gateway

Stack / practices: Java 21, Resilience4j, virtual threads, Spring Cloud Gateway, Micrometer, Prometheus

- Designed a **reusable resilience library** with a single public operation that composes **retry → time limiter → circuit breaker → bulkhead** in that order, so retries fit inside the time budget and the breaker sees the final outcome, not each attempt.
- Treated **HTTP 429 as backpressure rather than failure**, to avoid opening the circuit exactly when the remote system is asking us to slow down.
- Modelled the policy as a type that **validates its own configuration on construction**, so an inconsistent setup fails at startup rather than in production; exported per-instance metrics to Micrometer.
- Also built a **dedicated gateway with 7 named circuit breakers** and two policy profiles (normal and long-running operations) that isolates the legacy systems, with an HTTP fallback that answers Retry-After instead of letting the request die.
- Covered the behaviour with **integration tests that exercise the gateway with the upstream down**.

6. Asynchronous bulk document processing

Stack / practices: Spring Boot 4.1, Spring Cloud Stream, RabbitMQ, AWS S3, PostgreSQL, virtual threads, JDBC

- Moved thousand-document package uploads off the synchronous path: the producer stores the package in object storage and publishes **only the event**, answering the client immediately.
- Implemented **idempotency persisted in the database**, so a retry never duplicates what was already confirmed and does retry what failed — the only real "exactly once" guarantee is an idempotent consumer.
- Configured a **dead-letter queue with republishing**, three attempts, bounded concurrency and prefetch: 1 so a slow consumer cannot hoard messages, plus a function that drains the DLQ and a **scheduled technical requeue**.
- Processed in **bounded chunks (1,000 documents)**, republishing the event with the remainder so no single message can exhaust the acknowledgement window.
- Exposed an internal requeue endpoint for operations and kept per-chunk traceability in the database.

7. Domain services: billing, plans, purchases and documents

Stack / practices: Spring Boot 4, JPA/Hibernate, R2DBC, WebFlux, MapStruct, Apache POI, JasperReports, AWS S3 and SES

- Built the **billing and per-customer plan management service**, the largest on the platform: **118 REST endpoints, 30 entities**, bulk invoice generation, a billing dashboard and batch processing.
- Built the **reactive electronic purchase-records service** with WebFlux and **R2DBC**, using functional routers and persistence across **two engines (PostgreSQL and SQL Server)**.
- Implemented the configuration and master-data service that acts as the source of truth for the gateways, and the corporate mail service on **AWS SES**, also consumed by the identity provider.
- Contributed to document generation (reports, PDF and QR codes) and spreadsheet export with Apache POI.

8. Micro-frontend platforms with Nx and Native Federation

Stack / practices: Angular 21, Nx 22, Native Federation, Signals, standalone components, TailwindCSS 4, keycloak-js

- Worked on a monorepo with a host shell and **10 domain-federated remotes** (documents, traceability, analytics, billing, identity, company administration, reports, settings and purchases) plus 6 shared libraries.
- Defined a **navigation contract**: each remote exposes its menu items and the shell merges, de-duplicates and **filters them by the user's entitlements** computed from the token — nobody edits a central menu.
- Built the **shared authentication-context library** with reactive state (signals), exposing permission queries instead of each remote interpreting claims on its own.
- Shared dependencies as singletons with strict versions to avoid duplicating the framework in the bundle, and compiled CSS per remote on a common utility base.
- Also built two standalone Angular 21 applications (administration and documents) with test coverage wired into SonarQube.

9. Commercial transactional solution: stock control, auditing and integration events

Stack / practices: Java 21, Micronaut 4, Micronaut Security, Keycloak, PostgreSQL, Flyway, HikariCP, Kafka, Docker, GraalVM Native Image, Angular 21

- Backend for commercial processes with transactional consistency across operations, stock, auditing, traceability and integration events.
- Designed business rules to control states, validate availability, prevent invalid operations and keep integrity across critical entities.
- Implemented operational traceability through authenticated-user auditing, timestamps, functional states and critical movement logging.
- Applied the Outbox Pattern to decouple the main transaction from later event publishing, preserving consistency between database and messaging.
- Implemented scheduled publishing, retries, PENDING/PUBLISHED/ERROR state control and readiness for event-driven integration with Kafka.
- Configured security with JWT/JWKS, role-based authorisation, public/private routes, Micronaut Data JDBC, HikariCP, Flyway and Docker/Native Image readiness.

10. Credit application system with BPM and automated decisioning

Stack / practices: Java 21, Spring Boot 3, Camunda BPM, PostgreSQL, Angular 17+, hexagonal architecture

- Enterprise solution to digitise the credit application evaluation, approval and follow-up cycle.
- Built the backend on hexagonal architecture, separating domain, BPM orchestration, persistence, integrations and technical adapters.
- Implemented BPMN processes with automated risk evaluation rules, conditional decisions, SLAs and automatic escalation.
- Integrated HTML notifications, events and state updates to improve traceability and visibility of the operational cycle.
- Good practices: decoupling process from domain, state traceability, error control, testing and separation of responsibilities.

11. BPMN orchestration with Camunda 8 Self-Managed for order management

Stack / practices: Java 21, Spring Boot 4, Camunda 8, Zeebe, PostgreSQL, REST API

- Orchestration solution to manage orders through business states, external events and task automation.
- Built a REST API to start processes, handle business events and persist order traceability in PostgreSQL.
- Configured message correlation to advance flows from external systems, and workers for automation, validation and process closure.
- Decoupled business logic, persistence and the BPMN engine, validating the flow from the API, the engine and operational monitoring.

12. Port operations optimisation — Terminal Portuarios Peruanos (TPP)

Stack / practices: Spring Boot 3, Angular 19, AWS SDK, MyBatis, SQL Server, SonarQube, JUnit 5, JaCoCo

- Solution to optimise port operations and control through early validation, automation and traceability.
- Implemented cut-off validations to prevent improper entries into the operational flow and improve upfront control of critical operations.
- Built automated pre-appointments, dynamic QR codes and periodic operational alerts to improve follow-up and efficiency.
- Contributed to improvements that reduced operational errors by roughly 40%, reinforcing process stability and quality.
- Good practices: unit testing, static analysis, coverage, technical documentation and controlled evolutionary maintenance.

13. VUCE 2.0 — Single-window foreign trade platform and port processes

Stack / practices: Spring Boot 3, Java 17, Quarkus, MapStruct, JasperReports, Gradle, Lombok, Angular 17, Redis, Kafka, Feign, Camunda BPM

- Digital transformation solution for foreign trade processes, port operations, case traceability and document automation.
- Designed contract-first microservices with OpenAPI and hexagonal architecture, adding an experience layer to improve organisation, scalability and maintainability.
- Implemented inbound/outbound vessel processes, case tracking, bulk uploads and technical integration documentation.
- Integrated digital signature for port clearance, JasperReports reporting, a payment gateway, Redis, Kafka, Feign and Camunda BPM.
- Contributed sequence diagrams, contracts, functional/technical testing and reusable frontend libraries for Angular standardisation.

Sagone - Techera

Java Developer | October 2018 - April 2024

Development, evolution and integration of enterprise solutions for regulated industries and organisations running continuous operations. Focus on Java backend, BPMN processes, document integration, interoperability, performance, stability and evolutionary maintenance, including Android mobile development.

14. Electricity Transmission System (STE) — OSINERGMIN

Stack / practices: Java, Spring Boot, Quarkus, Camunda 7, Redis, relational databases

- Regulatory solution for technical and economic processes of the electricity transmission system, supporting planning, valuation and regulatory control.
- Led the implementation of BPMN processes with Camunda 7 to automate regulatory and operational flows of the electricity business.
- Integrated BPMN processes, backend services and persistence to ensure traceability, controlled execution, state transitions and process follow-up.
- Developed business rules, calculations, validations, queries and processing of technical, economic and regulatory information.
- Optimised load and query processes using Redis as a cache for temporary file-related information and intensive operations.
- Ensured operational continuity through evolutionary and corrective maintenance of critical systems aligned to regulatory change.

15. FACILITO (OSINERGMIN) — Android mobile application

Stack / practices: Android Studio, Android SDK, Java for Android, REST API consumption, geolocation, error handling, evolutionary maintenance and mobile UX

- Contributed to the development and evolutionary maintenance of FACILITO, a free public mobile app that lets citizens locate and compare fuel and gas prices at registered outlets.
- Implemented and refined mobile features for information lookup, navigation, filters, listings, outlet location and up-to-date price display from backend services.
- Integrated REST API consumption and response handling to present reliable information to end users, with validation, error control and support for intermittent connectivity.
- Applied Android good practice: separation of responsibilities, maintainability, cross-device compatibility, data usage optimisation and attention to user experience.
- Contributed to the stabilisation, support and continuous improvement of a high-usage citizen app, prioritising performance, incident traceability and operational continuity.

16. Document management, interoperability and enterprise platform evolution

Stack / practices: Java EE, Spring Boot, Quarkus, Angular, React, Alfresco, REST/SOAP APIs, Kafka, AWS, Docker, Kubernetes, Jenkins, GitLab CI/CD, Atlassian

- **Document management integrated with Alfresco:** built integrations for upload, download, search, document classification and metadata administration, keeping the transactional system and the document repository consistent through access rules, traceability and a clear separation between document integration and business logic.
- **Interoperability with national identity services and external providers:** implemented connectors, validation, response adaptation and error handling across heterogeneous platforms, separating external contracts, data mapping and internal rules to reduce coupling and ease maintenance.
- **CRM, e-commerce, education and operational control platforms:** built customer management, sales follow-up, catalogues, purchase flow, academic records, stock and operational traceability, contributing performance improvements, incident fixes and cross-layer integration.
- **Evolutionary maintenance and operational continuity for more than 50 production systems** of varying criticality: incident fixes, functional adjustments and adaptation to new requirements without compromising stability, collaborating on progressive modernisation with microservices, hexagonal architecture and CI/CD automation.

Solución

Junior Developer / Intern | October 2015 - October 2017

- Started my career building enterprise applications with JSP, Servlets, MVC, DAOs and SQL queries.
- Supported application maintenance, technical support, functional QA and technical testing.

CROSS-CUTTING GOOD PRACTICES

- **Architecture:** separation of concerns, domain design, ports/adapters, OpenAPI contracts, clean layers and clear boundaries between services.
- **Security:** OAuth2/OIDC/JWT, Keycloak, MFA, SPIs, RBAC, token validation, claim normalisation, user auditing and role/group control.
- **Resilience:** circuit breaker, retry, bulkhead, time limiter, fallbacks, timeouts, backpressure and isolation of legacy dependencies.
- **Integration:** Outbox Pattern, retries, asynchronous publishing, idempotency, dead-letter queues, Kafka, RabbitMQ, Redis, Feign, BPMN and N8N automation.
- **Quality:** JUnit 5, Mockito, JaCoCo, SonarQube, global error handling, standard responses, centralised validation and duplication reduction.
- **Operations:** traceability, logging, Micrometer/Prometheus metrics, health checks, documentation, sequence diagrams, database versioning and Docker/Kubernetes.

EDUCATION

- **B.Eng. Systems & Computer Engineering** — Universidad Tecnológica del Perú (UTP) · 2024 – in progress
- **Professional Degree, Telematics** — IDAT · 2015 – 2017

LANGUAGES

- **Spanish:** native
- **English:** intermediate — technical reading and writing